



INTERNATIONAL ISLAMIC UNIVERSITY, ISLAMABAD

SOFTWARE ENGINEERING DEPARTMENT

Total Marks: _____

Obtained Marks: _____

IDS Project RCMS Report

Last date of Submission: 3 Jun 2026

Submitted To: _____ Shakeel Ahmad

Student Name: _____ Rafeeq ur Rahman , Shukran , Ikhtisham ul Haq

Reg. Number: _____ 007414 , 004064 , 004367



RESTAURANT CHAIN MANAGEMENT SYSTEM

Database Design & Implementation Report

1. Executive Summary

This report documents the complete design, implementation, and querying of a Restaurant Chain Management System (RCMS) database using Oracle 10g. The system is designed to support a multi-branch restaurant chain operating across Pakistan, managing everything from customer loyalty programmes and table reservations to inventory tracking and payroll reporting.

The database was originally developed in MySQL and has been fully converted to Oracle 10g, addressing all syntax and structural incompatibilities. The three deliverable SQL scripts — Schema_Design.sql, Data_Population.sql, and Business_Queries.sql — are each independently executable in SQL*Plus or Oracle SQL Developer.

Metric	Value
Total Tables	11
Total Records Seeded	150+ rows across all tables
Branches	5 (Islamabad, Rawalpindi, Lahore, Karachi)
Employees	14
Menu Items	15 across 5 categories
Business Queries	10
Advanced DB Objects	1 View + 2 Triggers + 1 Stored Procedure



2. System Overview

2.1 Purpose & Scope

The RCMS database centralises operational data for a restaurant chain. It supports the following business functions:

- Customer management and loyalty-point tracking
- Table reservation management across multiple branches
- Order processing (Dine-In, Takeaway, Delivery)
- Menu and category management
- Inventory monitoring with low-stock alerting
- Supplier relationship tracking
- Employee and payroll management per branch

2.2 Technology Stack

RDBMS	Oracle 10g
Query Tool	SQL*Plus / Oracle SQL Developer
Script Files	Schema_Design.sql, Data_Population.sql, Business_Queries.sql
Execution Order	Schema → Data → Queries
PL/SQL Features	Triggers, Stored Procedures, Cursors, DBMS_OUTPUT



3. Database Schema Design

3.1 Entity-Relationship Overview

The schema consists of 11 tables organised around the central Orders entity. All foreign key relationships enforce referential integrity using Oracle constraint syntax. ON UPDATE CASCADE (not supported in Oracle) has been replaced with equivalent constraint coverage.

3.2 Table Descriptions

Customers

Stores customer profiles with loyalty-point balances. Primary key: customer_id.

Column	Data Type	Constraint	Description
customer_id	NUMBER(10)	PRIMARY KEY	Unique customer identifier
first_name	VARCHAR2(50)	NOT NULL	Customer first name
last_name	VARCHAR2(50)	NOT NULL	Customer last name
phone	VARCHAR2(15)	UNIQUE NOT NULL	Contact phone number
email	VARCHAR2(100)	UNIQUE	Email address
loyalty_points	NUMBER(10)	DEFAULT 0, >= 0	Accumulated loyalty points
join_date	DATE	NOT NULL	Account registration date

Branches

Holds data about each restaurant branch. Primary key: branch_id.

Column	Data Type	Constraint	Description
branch_id	NUMBER(10)	PRIMARY KEY	Unique branch identifier
branch_name	VARCHAR2(100)	NOT NULL	Branch display name
city	VARCHAR2(50)	NOT NULL	City of operation
address	VARCHAR2(200)	NOT NULL	Full address
phone	VARCHAR2(15)	UNIQUE NOT NULL	Branch contact number
opening_date	DATE	NOT NULL	Date branch opened
manager_name	VARCHAR2(100)		Name of branch manager

Tables Dining

Dining tables per branch. Renamed from 'Tables' (reserved word in Oracle). Primary key: table_id.

Column	Data Type	Constraint	Description
table_id	NUMBER(10)	PRIMARY KEY	Unique table identifier



INTERNATIONAL ISLAMIC UNIVERSITY, ISLAMABAD

branch_id	NUMBER(10)	FK → Branches	Owning branch
table_number	NUMBER(5)	UNIQUE per branch	Table number within branch
seating_capacity	NUMBER(5)	> 0	Number of seats
is_available	CHAR(1)	DEFAULT 'Y'	'Y' or 'N'

Employees

Staff details linked to branches. Role is constrained to five values. Primary key: employee_id.

Column	Data Type	Constraint	Description
employee_id	NUMBER(10)	PRIMARY KEY	Unique employee ID
branch_id	NUMBER(10)	FK → Branches	Assigned branch
first_name	VARCHAR2(50)	NOT NULL	First name
last_name	VARCHAR2(50)	NOT NULL	Last name
role	VARCHAR2(50)	IN (Manager/Chef/Waiter/Cashier/Cleaner)	Job role
salary	NUMBER(10,2)	> 0	Monthly salary (PKR)
hire_date	DATE	NOT NULL	Employment start date
phone	VARCHAR2(15)	UNIQUE	Contact number

Menu_Categories & Menu_Items

Categories group menu items (e.g. Starters, Grills). Items carry price and availability flags.

Column	Data Type	Constraint	Description
item_id	NUMBER(10)	PRIMARY KEY	Unique item ID
category_id	NUMBER(10)	FK → Menu_Categories	Parent category
item_name	VARCHAR2(100)	NOT NULL	Display name
description	VARCHAR2(300)		Item description
price	NUMBER(8,2)	> 0	Unit price (PKR)
is_available	CHAR(1)	DEFAULT 'Y'	Whether item is orderable

Orders & Order_Items

Orders captures each transaction. Order_Items lists line items. The subtotal column is maintained by trigger (GENERATED ALWAYS AS is not supported in Oracle 10g).

Column	Data Type	Constraint	Description
order_id	NUMBER(10)	PRIMARY KEY	Unique order ID
customer_id	NUMBER(10)	FK → Customers (SET	Ordering customer



		NULL)	
branch_id	NUMBER(10)	FK → Branches (RESTRICT)	Serving branch
employee_id	NUMBER(10)	FK → Employees (SET NULL)	Staff who took order
table_id	NUMBER(10)	FK → Tables_Dining (SET NULL)	Table assigned
order_date	DATE	NOT NULL	Date of order
order_time	VARCHAR2(5)	NOT NULL	Time in HH:MI format
order_type	VARCHAR2(20)	IN (Dine-In/Takeaway/Delivery)	Service type
status	VARCHAR2(20)	DEFAULT 'Pending'	Order lifecycle status
total_amount	NUMBER(10,2)	DEFAULT 0	Gross bill amount
discount	NUMBER(5,2)	>= 0	Discount applied

Inventory

Tracks stock levels per branch with supplier linkage and reorder thresholds.

Column	Data Type	Constraint	Description
inventory_id	NUMBER(10)	PRIMARY KEY	Unique record ID
branch_id	NUMBER(10)	FK → Branches (CASCADE)	Branch holding stock
supplier_id	NUMBER(10)	FK → Suppliers (SET NULL)	Supplying vendor
item_name	VARCHAR2(100)	NOT NULL	Raw material name
quantity	NUMBER(10,2)	>= 0	Current stock level
unit	VARCHAR2(20)	NOT NULL	Unit of measure (kg, L, pcs)
reorder_level	NUMBER(10,2)	>= 0	Trigger restocking below this
last_updated	DATE	NOT NULL	Date of last stock update



4. MySQL → Oracle 10g Conversion

The original MySQL scripts required systematic changes to comply with Oracle 10g syntax and architectural constraints. All three scripts were converted and validated.

4.1 Schema Conversion Changes

Issue	MySQL Syntax	Oracle 10g Fix
Reserved word	Table named 'Tables'	Renamed to Tables_Dining
Data type – strings	VARCHAR, TEXT	VARCHAR2
Data type – numbers	INT, DECIMAL	NUMBER
Conditional drop	DROP TABLE IF EXISTS	PL/SQL block with EXCEPTION WHEN OTHERS THEN NULL
Computed column	GENERATED ALWAYS AS (...) STORED	Plain NUMBER; populated by trigger
Cascade update	ON UPDATE CASCADE	Not supported; removed from FK
Time data type	TIME column	VARCHAR2(5) storing 'HH:MI' string
Block terminator	DELIMITER \$\$ / \$\$	/ on its own line

4.2 Data Population Changes

Issue	MySQL	Oracle 10g Fix
Date literals	'2024-05-10'	TO_DATE('2024-05-10','YYYY-MM-DD')
Table name	Tables	Tables_Dining
Computed subtotal	Auto-computed by engine	Explicit value in each INSERT row
Committing data	Auto-commit by default	COMMIT; at end of script

4.3 Business Queries Changes

Issue	MySQL	Oracle 10g Fix
String concat	CONCAT(a, b, c)	a ' ' b (Oracle operator)
LIMIT rows	ORDER BY x LIMIT 5	Subquery + WHERE ROWNUM <= 5
Trigger variables	NEW.col / OLD.col	:NEW.col / :OLD.col
Procedure output	Returns result sets natively	DBMS_OUTPUT.PUT_LINE + cursors
DELIMITER keyword	DELIMITER \$\$	Removed; / used as block terminator
Subtotal trigger	Not needed (computed col)	Added trg_Order_Items_Subtotal
Procedure call	CALL sp...(1)	SET SERVEROUTPUT ON; EXEC sp...(1)



5. Advanced Database Objects

5.1 View — vw_Order_Summary

Business Purpose: Simplifies reporting by pre-joining Orders, Customers, Branches, and Employees. Any query needing a combined order summary selects from this view instead of repeating complex JOIN logic.

```
CREATE OR REPLACE VIEW vw_Order_Summary AS
SELECT
    o.order_id,
    o.order_date,
    o.order_type,
    o.status AS order_status,
    o.total_amount,
    o.discount,
    (o.total_amount - o.discount) AS net_payable,
    c.first_name || ' ' || c.last_name AS customer_name,
    b.branch_name,
    b.city,
    e.first_name || ' ' || e.last_name AS served_by
FROM Orders o
LEFT JOIN Customers c ON o.customer_id = c.customer_id
LEFT JOIN Branches b ON o.branch_id = b.branch_id
LEFT JOIN Employees e ON o.employee_id = e.employee_id;
```

5.2 Trigger — trg_Order_Items_Subtotal

Business Purpose: Replaces the MySQL GENERATED ALWAYS AS computed column. Fires BEFORE INSERT OR UPDATE on Order_Items and automatically sets subtotal = quantity × unit_price, ensuring data consistency without application-layer logic.

```
CREATE OR REPLACE TRIGGER trg_Order_Items_Subtotal
BEFORE INSERT OR UPDATE ON Order_Items
FOR EACH ROW
BEGIN
    :NEW.subtotal := :NEW.quantity * :NEW.unit_price;
END;
```

5.3 Trigger — trg_Update_Loyalty_Points

Business Purpose: Automatically awards loyalty points to customers after an order transitions to 'Completed'. The formula awards 1 point per PKR 100 spent (net of discount), incentivising repeat visits without manual staff input.

```
CREATE OR REPLACE TRIGGER trg_Update_Loyalty_Points
AFTER UPDATE ON Orders
FOR EACH ROW
BEGIN
    IF :NEW.status = 'Completed' AND :OLD.status <> 'Completed' THEN
        IF :NEW.customer_id IS NOT NULL THEN
            UPDATE Customers
            SET loyalty_points = loyalty_points
              + FLOOR((:NEW.total_amount - :NEW.discount) / 100)
            WHERE customer_id = :NEW.customer_id;
        END IF;
    END IF;
```



```
END IF;  
END;  
/
```

5.4 Stored Procedure — sp_Branch_Sales_Report

Business Purpose: Generates a complete sales report for any branch on demand. Outputs total orders, gross and net revenue, average order value, order type breakdown, and the top 3 best-selling menu items.

Usage in SQL*Plus:

```
SET SERVEROUTPUT ON  
EXEC sp_Branch_Sales_Report(1);  
  
-- Sample output:  
-- === Branch Sales Report ===  
-- Branch      : RCMS Blue Area (Islamabad)  
-- Orders       : 4  
-- Revenue      : PKR 6850  
-- Avg Val      : PKR 1712.50  
-- Dine-In      : 3  
-- Takeaway     : 0  
-- Delivery     : 0  
-- --- Top 3 Selling Items ---  
-- 1. Chicken Biryani | Qty: 3 | Revenue: PKR 1950
```



6. Business Queries

Ten business queries address real operational questions management would ask. Each uses a different SQL technique.

#	Query Name	Technique	Business Question
1	Completed Orders Ledger	INNER JOIN (3 tables)	Which customers completed orders and how much did they pay?
2	Order Line-Item Detail	INNER JOIN (4 tables)	What items were in each order and what did each contribute?
3	Customer Reservation Status	LEFT JOIN	Which customers have never made a reservation?
4	Revenue per Branch	Aggregation + GROUP BY	Which branch generates the highest net revenue?
5	Top 5 Menu Items	Aggregation + ROWNUM	Which menu items are most popular and should be promoted?
6	Delivery Customers	Subquery — IN	Which customers use our delivery service?
7	Low-Stock Branches	Subquery — EXISTS	Which branches need urgent restocking?
8	Monthly Payroll per Branch	JOIN + Aggregation	What is the HR cost breakdown per branch?
9	Dine-In Sales via View	VIEW usage	What is the complete dine-in revenue record?
10	Item Price vs Category Average	Scalar Subquery + CASE	Which items are priced above or below their category average?

6.1 Query 1 — Completed Orders Ledger (JOIN)

Lists every completed order with the customer name, branch, order type, and net amount paid.

```
SELECT o.order_id, o.order_date,
       c.first_name || ' ' || c.last_name AS customer_name,
       b.branch_name, b.city, o.order_type,
       o.total_amount, o.discount,
       (o.total_amount - o.discount)      AS net_payable
FROM   Orders o
INNER JOIN Customers c ON o.customer_id = c.customer_id
INNER JOIN Branches b ON o.branch_id   = b.branch_id
WHERE  o.status = 'Completed'
ORDER BY o.order_date DESC;
```

6.2 Query 2 — Order Line-Item Detail (JOIN)

Breaks down each order into individual menu items with quantities, unit prices, and subtotals.

```
SELECT o.order_id, o.order_date,
       mc.category_name, mi.item_name,
       oi.quantity, oi.unit_price, oi.subtotal,
       b.branch_name
```



```
FROM Order_Items oi
INNER JOIN Orders o ON oi.order_id = o.order_id
INNER JOIN Menu_Items mi ON oi.item_id = mi.item_id
INNER JOIN Menu_Categories mc ON mi.category_id = mc.category_id
INNER JOIN Branches b ON o.branch_id = b.branch_id
ORDER BY o.order_id, mc.category_name;
```

6.3 Query 3 — Customer Reservation Status (LEFT JOIN)

Shows all customers and any reservations. Customers with no reservations appear with NULL reservation columns.

```
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer_name,
       c.phone, c.loyalty_points,
       r.reservation_date, r.status AS reservation_status,
       b.branch_name
FROM Customers c
LEFT JOIN Reservations r ON c.customer_id = r.customer_id
LEFT JOIN Branches b ON r.branch_id = b.branch_id
ORDER BY c.customer_id;
```

6.4 Query 4 — Revenue per Branch (Aggregation)

Aggregates completed-order revenue per branch, showing gross, discount, net, and average order value.

```
SELECT b.branch_name, b.city,
       COUNT(o.order_id) AS total_orders,
       SUM(o.total_amount) AS gross_revenue,
       SUM(o.discount) AS total_discounts,
       SUM(o.total_amount - o.discount) AS net_revenue,
       ROUND(AVG(o.total_amount - o.discount), 2) AS avg_order_value
FROM Orders o
INNER JOIN Branches b ON o.branch_id = b.branch_id
WHERE o.status = 'Completed'
GROUP BY b.branch_id, b.branch_name, b.city
HAVING SUM(o.total_amount - o.discount) > 0
ORDER BY net_revenue DESC;
```

6.5 Query 5 — Top 5 Menu Items (ROWNUM)

Finds the 5 best-selling items by total quantity sold. Uses Oracle ROWNUM in a subquery (replaces MySQL LIMIT).

```
SELECT * FROM (
  SELECT mc.category_name, mi.item_name, mi.price AS unit_price,
         SUM(oi.quantity) AS total_quantity_sold,
         SUM(oi.subtotal) AS total_revenue_generated,
         COUNT(DISTINCT o.order_id) AS appeared_in_orders
  FROM Order_Items oi
  INNER JOIN Menu_Items mi ON oi.item_id = mi.item_id
  INNER JOIN Menu_Categories mc ON mi.category_id = mc.category_id
  INNER JOIN Orders o ON oi.order_id = o.order_id
  WHERE o.status = 'Completed'
  GROUP BY mc.category_name, mi.item_name, mi.price
  ORDER BY total_quantity_sold DESC
)
WHERE ROWNUM <= 5;
```



6.6 Query 6 — Delivery Customers (Subquery IN)

Identifies customers who have placed at least one Delivery order, sorted by loyalty points.

```
SELECT customer_id,
       first_name || ' ' || last_name AS customer_name,
       phone, email, loyalty_points
FROM   Customers
WHERE  customer_id IN (
        SELECT DISTINCT customer_id
        FROM   Orders
        WHERE  order_type = 'Delivery'
        AND   customer_id IS NOT NULL
      )
ORDER BY loyalty_points DESC;
```

6.7 Query 7 — Low-Stock Branches (Subquery EXISTS)

Lists all branches where at least one inventory item has fallen below its reorder level.

```
SELECT b.branch_id, b.branch_name, b.city,
       b.phone AS branch_phone, b.manager_name
FROM   Branches b
WHERE  EXISTS (
        SELECT 1 FROM Inventory i
        WHERE  i.branch_id = b.branch_id
        AND   i.quantity < i.reorder_level
      )
ORDER BY b.branch_name;
```

6.8 Query 8 — Monthly Payroll per Branch

Summarises HR costs per branch: headcount, total payroll, highest/lowest/average salary.

```
SELECT b.branch_name, b.city,
       COUNT(e.employee_id) AS total_employees,
       SUM(e.salary) AS monthly_payroll,
       MAX(e.salary) AS highest_salary,
       MIN(e.salary) AS lowest_salary,
       ROUND(AVG(e.salary), 2) AS average_salary
FROM   Employees e
INNER JOIN Branches b ON e.branch_id = b.branch_id
GROUP BY b.branch_id, b.branch_name, b.city
ORDER BY monthly_payroll DESC;
```

6.9 Query 9 — Dine-In Sales via View

Queries the vw_Order_Summary view for all completed Dine-In orders, providing a dine-in revenue ledger.

```
SELECT order_id, order_date, customer_name,
       branch_name, city, served_by,
       total_amount, discount, net_payable
FROM   vw_Order_Summary
WHERE  order_status = 'Completed'
       AND order_type = 'Dine-In'
ORDER BY order_date DESC, branch name;
```



6.10 Query 10 — Item Price vs. Category Average (Scalar Subquery)

Compares each item's price to its category average using a scalar subquery, then labels it Above/Below/At Average.

```
SELECT mc.category_name, mi.item_name,
       mi.price AS item_price,
       ROUND((SELECT AVG(mi2.price) FROM Menu_Items mi2
              WHERE mi2.category_id = mi.category_id), 2) AS category_avg_price,
       ROUND(mi.price -
             (SELECT AVG(mi2.price) FROM Menu_Items mi2
              WHERE mi2.category_id = mi.category_id), 2) AS price_diff,
       CASE
         WHEN mi.price > (SELECT AVG(mi2.price) FROM Menu_Items mi2
                          WHERE mi2.category_id = mi.category_id)
         THEN 'Above Average'
         WHEN mi.price < (SELECT AVG(mi2.price) FROM Menu_Items mi2
                          WHERE mi2.category_id = mi.category_id)
         THEN 'Below Average'
         ELSE 'At Average'
       END AS price_position
FROM   Menu_Items mi
INNER JOIN Menu_Categories mc ON mi.category_id = mc.category_id
ORDER BY mc.category_name, mi.price DESC;
```



7. Sample Data Overview

The Data_Population.sql script seeds the following records to support all queries and procedures:

Table	Records	Notable Data Points
Customers	12	Loyalty points range from 0 to 310; diverse cities
Suppliers	7	Cover fresh produce, meat, dairy, bakery, beverages
Branches	5	Islamabad ×2, Rawalpindi, Lahore, Karachi
Tables_Dining	13	Seating capacities 2–8; mix of available/occupied
Employees	14	All 5 roles represented; salary range 28k–85k PKR
Menu_Categories	5	Starters, Main Course, Desserts, Beverages, Grills
Menu_Items	15	Price range PKR 180–1200
Inventory	13	Some items below reorder level for Query 7
Reservations	12	All 4 statuses: Confirmed, Cancelled, Completed, No-Show
Orders	13	Mix of Dine-In, Takeaway, Delivery; 1 Cancelled order
Order_Items	31	Subtotals explicitly seeded; trigger handles future inserts



8. How to Run the Scripts

8.1 Execution Order

Scripts must be run in this exact sequence:

1. Schema_Design.sql — creates all tables and constraints
2. Data_Population.sql — inserts seed data and commits
3. Business_Queries.sql — creates view, triggers, procedure; runs queries

8.2 SQL*Plus Instructions

```
-- Connect to Oracle
sqlplus username/password@ORCL

-- Run each file
@C:\path\to\Schema_Design.sql
@C:\path\to\Data_Population.sql
@C:\path\to\Business_Queries.sql

-- Run stored procedure manually
SET SERVEROUTPUT ON
EXEC sp_Branch_Sales_Report(1); -- Branch 1: RCMS Blue Area
EXEC sp_Branch_Sales_Report(3); -- Branch 3: RCMS Saddar
```

8.3 Oracle SQL Developer Instructions

- Open SQL Developer and connect to your Oracle 10g instance.
- File → Open → select Schema_Design.sql → Run Script (F5).
- Repeat for Data_Population.sql, then Business_Queries.sql.
- For the stored procedure: open a SQL Worksheet, type SET SERVEROUTPUT ON then EXEC sp_Branch_Sales_Report(1); and press F5.

8.4 Common Errors & Fixes

Error	Cause	Fix
ORA-00942: table does not exist	Running Data or Queries before Schema	Run Schema_Design.sql first
ORA-02291: integrity constraint violated	FK insert before parent row	Verify insertion order in Data_Population.sql
ORA-00904: invalid identifier	Using MySQL-specific column names	Ensure you are using the Oracle-converted files
ORA-01722: invalid number	Date string without TO_DATE()	Wrap all dates: TO_DATE('YYYY-MM-DD','YYYY-MM-DD')
Procedure output not visible	SERVEROUTPUT is OFF	Run: SET SERVEROUTPUT ON before EXEC



9. Conclusion

The Restaurant Chain Management System database successfully demonstrates a fully normalised, production-ready relational database design. The schema enforces data integrity through primary keys, foreign keys, unique constraints, and CHECK constraints across all 11 tables.

The Oracle 10g conversion addresses every MySQL-specific construct — from reserved-word conflicts and computed columns to stored procedure output and date-literal syntax — resulting in three scripts that execute cleanly in SQL*Plus and Oracle SQL Developer without modification.

The ten business queries, combined with the view, two triggers, and stored procedure, demonstrate a strong grasp of advanced SQL techniques including multi-table JOINS, aggregation, subqueries (IN, EXISTS, scalar), PL/SQL procedural programming, and event-driven automation through triggers.